

AGENTMPI: A Message Passing Interface for Multi-Agent Systems

Anonymous submission

(artefact: protocol specification, reference runtime, experiment harnesses)

Abstract

Multi-agent LLM harnesses are written the way parallel programs were written in 1991: every group re-implements process naming, work distribution, information sharing, synchronisation and failure handling, incompatibly, inside its application. The historical answer to that situation was not a better framework but a *standardised interface* — MPI — which succeeded by specifying mechanisms rather than policy and by leaving meaning to the application. We ask whether that answer transfers, and find that it largely does, but that the *cost model* does not, and rederiving it changes which algorithms are correct to use.

We present AGENTMPI, a message passing interface for multi-agent systems. It keeps MPI’s structure — communicators with private contexts, matched point-to-point communication with a non-overtaking guarantee, the full collective catalogue, one-sided windows, and ULFM’s revoke/shrink/agree fault-tolerance primitives — and rederives its cost model around the three properties that distinguish an LLM agent from a process: its context window, not memory bandwidth, is the scarce resource; applying a reduction operator costs minutes rather than nanoseconds; and it fails individually, frequently, and sometimes silently. Three mechanisms follow. *Context-denominated flow control* reinterprets MPI’s eager limit in tokens, so a large payload is delivered as a handle the receiver may decline to read; measured at $P = 128$, this reduces the peak tokens in a single agent’s context during an allgather from 501,888 to 3200, a factor of 157. *Agent-evaluated operators* make `MPI_Op_create`’s callback a language model through a continuation protocol, which makes the number of operator applications on the critical path the dominant cost and thereby *inverts* MPI’s selection rules: recursive-doubling allreduce, MPI’s standard short-message choice, moves $36.3 \times$ more payload than reduce-then-broadcast at $P = 128$. *Two-phase lease-based detection* with fencing epochs and a join deadline makes launch failure, silent stalls and zombies detectable without a monitoring daemon.

We evaluate a reference runtime with microbenchmarks to $P = 128$ and with multi-agent runs in which every rank is a separate LLM agent coordinating only through the protocol. Two applications are studied:

parallel book translation, an apparently embarrassingly parallel task whose real dependencies are a global terminology agreement and a nearest-neighbour boundary agreement; and the collaborative implementation of a Scheme interpreter by eight agents, graded by a 174-case held-out suite. The translation task shows the protocol eliminating terminology divergence at a tenth of the peak context; on the software task both arms passed every held-out case, so we report that null result in full: the arm we ablated spontaneously recreated the protocol’s own mechanisms — unprompted, its agents built a contract window and per-rank interface declarations — and reached the same score for a third of the cost. Differential testing over 2500 generated programs then discriminated where the suite could not, at 92.4 % agreement: all 111 specification-attributable disagreements favour the protocol arm, tracing to 1 defect that arose in both arms and that only the protocol arm’s synchronised integration round removed. A run in which 16 of 22 requested ranks never started completed with all 6 survivors finalised and every collective closed. We also report the fifteen protocol and harness defects that only real agents exposed — among them that an agent instructed to retry a timed-out call twenty times gave up after two, that a rank blocked in a barrier was declared dead for the crime of waiting, and that making rank identity ambient, which we had argued eliminated a class of errors, in fact replaced it with a worse one once the host’s shell sessions turned out to be shared.

1 Introduction

In 1992 a parallel application author faced a dozen mutually incompatible message passing libraries, each tied to one vendor’s machine, and paid the cost of that fragmentation twice: once writing a portability layer, and again on discovering that the layer hid the performance characteristics that mattered most [45, 20]. The response was the MPI Forum: roughly forty organisations, meeting openly, who produced not a framework but a *specification* [34]. MPI succeeded, in the retrospective judgement of its own designers, because it standardised what was already well understood, kept semantics thin, made data movement explicit, and — crucially — provided communicators, which gave libraries a private communication namespace and thereby made third-

party parallel libraries possible at all [20, 22, 16].

The situation in multi-agent LLM systems today is recognisably the same. Every harness re-implements agent naming, work partitioning, result aggregation, shared knowledge, synchronisation and failure recovery, inside its application and incompatibly with every other harness. The protocols that have emerged standardise adjacent problems: the Model Context Protocol standardises how one agent calls a *tool* [36]; Agent2Agent standardises how one agent delegates a *task* to another [1]. Neither provides group membership, collectives, consistency for shared state, or a failure model — which is to say, neither provides what MPI provides. Meanwhile the reported failure modes of real multi-agent systems read like a list of missing MPI features: agents cannot share what they learn, the system does not survive an agent’s death, there is no synchronisation discipline, and context windows overflow [7].

This paper asks whether MPI’s answer transfers. Our finding is that its *structure* transfers almost intact — we adopt communicators, matched messaging with non-overtaking, the collective catalogue, one-sided windows and ULFM [4] with little change — while its *cost model* does not, and that rederiving the cost model changes which algorithms are correct. The three differences that drive everything are:

The scarce resource is context, not bandwidth.

An MPI implementation’s eager limit exists because the receiver’s unexpected-message buffer is finite. An agent’s analogous buffer is its context window, which is smaller in the units that matter and cannot be enlarged at runtime.

Applying an operator costs minutes.

`MPI_Op_create` lets a user supply a reduction function; if that function is a language model, the cost of *one application* exceeds the cost of all communication in the collective by orders of magnitude.

Failure is individual, frequent and sometimes silent.

MPI’s default error handler aborts the job, which is defensible when a node fails weekly. Agents fail per task, in more ways, and often by returning a plausible wrong answer.

Contributions.

1. **A protocol.** AGENTMPI/0.1: a normative specification of a message passing interface for agents, deriving each departure from MPI from a stated property of agent executors (Section 4). It is a protocol for *writing* multi-agent systems, not a multi-agent system.
2. **Context-denominated flow control.** MPI’s eager/rendezvous split, denominated in tokens, plus *views*: declarative bounded projections of a payload that play the role of derived datatypes (Section 4.2).

3. **Agent-evaluated collectives and a rederived selection rule.** A continuation protocol that makes the reduction operator an agent, and the resulting selection rules, which differ from and in one case invert MPI’s (Section 5).
4. **A fault-tolerance design for unreliable executors.** Two-phase lease detection, fencing epochs, join deadlines, and a recovery briefing built by replaying committed commitments rather than restoring a memory image (Section 6).
5. **A reference runtime and an evaluation with real agents.** Microbenchmarks to $P = 128$ plus multi-agent runs in which every rank is a separate LLM agent (Section 8), including a mass-launch-failure episode and a catalogue of the protocol defects that only real agents exposed (Section 9).

2 Background: what MPI actually standardised

MPI’s own retrospectives identify a small number of decisions as load-bearing, and each has a direct bearing on our design.

A specification, not an implementation — with a reference implementation shipped alongside.

The Forum standardised semantics and left implementation free, but MPICH was not a post-hoc artefact: at the Forum’s organisational meeting Gropp and Lusk undertook to build an implementation tracking the standard *as it evolved*, specifically to expose problems before they were fixed in text [19]. HPF’s authors name the absence of such an implementation as a direct cause of HPF’s failure [28]. We follow this: the specification and the reference runtime in this paper were written together, and Section 9 lists the specification changes that running it forced.

Thin semantics. An MPI message is an opaque buffer plus a datatype. MPI has no ontology and no notion of what a message *means*. The contrast is with KQML [13] and FIPA-ACL [15], which specified performatives with mentalistic semantics (feasibility preconditions, rational effects) that proved unverifiable in principle and unnecessary in practice. AGENTMPI is deliberately on MPI’s side of this line: the protocol never interprets a payload.

Communicators. Every operation is scoped to a communicator carrying a private context, so a library operating on a subset of ranks cannot collide with its caller’s tags. IBM’s implementers, porting from EUI, singled this out as the significant change, precisely because a module on a subset of processes could then use a

local name space [16]; Gropp and Lusk state the corollary that makes it safe — the context must not be a visible value [22]. Nothing in the current agent ecosystem has an equivalent, which is why two agent “teams” in one harness share one global bus.

Explicit locality and a small core. MPI moves no data the programmer did not ask it to move. It has 128 routines in MPI-1 and 444 in MPI-3.1 [41, 31], yet six suffice for real programs [21], and a large-scale study found 42% of open-source MPI applications use only MPI-1.0 features [31]. Both properties — a small orthogonal core with a large convenience surface — are ones we imitate.

What MPI deliberately omitted, and what it assumed. The Forum left out fault tolerance, load balancing, and task scheduling as insufficiently understood. Two omissions are load-bearing for us. First, MPI’s stated goals *assume reliable communication*, so that “the user need not cope with communication failures” [35] — an assumption an agent harness cannot import. Second, PVM, which MPI displaced, was explicitly better at handling failures in loosely-coupled environments [22]: that is, better in AGENTMPI’s actual habitat. We therefore cannot simply adopt MPI-1’s posture, and we adopt ULFM’s instead.

3 Why an agent is not a process

Table 1 states the five differences that drive every departure in Section 4. They are properties of executors, not of any particular model or vendor.

The two planes. One structural observation organises the design. An AGENTMPI implementation has a *control plane* carrying envelopes, handles and synchronisation, which may be a shared medium every rank can read; and a *data plane* carrying payload bodies *into context windows*, which is irreducibly private and per-rank. Control collectives can therefore be $O(1)$ rounds — the analogue of in-network aggregation rather than of a software tree — while data collectives whose operator an agent must evaluate are bounded below by the serialised operator applications they require. Almost every quantitative result in Section 8 is a consequence of this split.

Cost model. For an operation moving n tokens,

$$T = \alpha + \beta n + \gamma k + \lambda, \quad (1)$$

where α is per-operation control-plane latency, β per-token control cost, γ the cost of *one operator application by an executor*, k the number of such applications on the critical path, and λ the executor’s own

think time (heavy-tailed). Equation (1) is the Hockney model [43] with two terms added. On our reference runtime $\alpha = 5.7 \text{ ms}$ and $\beta = 1.22 \mu\text{s}/\text{token}$ (Section 8.1), giving a half-bandwidth point of 4720 tokens, whereas γ and λ are seconds to minutes. The selection principle follows immediately: *MPI’s algorithm selection minimises α and βn and treats γ as free; AGENTMPI’s must minimise k and bound context cost.* Transplanting MPI’s rules unchanged gives wrong answers, and Section 8.1 gives a measured example.

4 Design

AGENTMPI is specified normatively elsewhere in the artefact; this section gives the design and its rationale. Table 2 summarises the correspondence.

4.1 Ranks, epochs and identity

A rank is a *role*; at most one executor occupies it at a time, identified by (r, e) where e is a monotone **epoch**. Any operation from a stale epoch fails with `AMPI_ERR_FENCED` and has no effect. The epoch is a fencing token, and it exists for the reason fencing tokens always exist: an executor cannot know that its lease expired mid-step, so between expiry and its next call two executors believe they own the rank. A lease alone makes that race unlikely; a token checked on every operation closes it [18, 6, 29].

Identity is *ambient* — taken from the environment rather than passed on every call — because in early testing the most frequent agent error by a wide margin was passing the wrong rank. That is necessary and, as Section 9 (L8) reports at length, it is not sufficient: ambient identity assumes the environment is trustworthy, and on a host that shares shell sessions between agents it is not. An implementation **MUST** therefore also let an executor *assert* its identity (`-expect-rank`, `-expect-job`) and fail before the operation runs, echo the acting identity on every operation, and check a launcher-issued per-rank token. Ambient by default, assertable on demand.

4.2 Context-denominated flow control

Every payload has an envelope (source, tag, token count, digest, optional schema, cheap deterministic summary) and a content-addressed handle. Let E be the eager threshold in tokens. A payload of $n \leq E$ tokens is delivered **eagerly**: the body enters the receiver’s context and the *context ledger* is charged n . A payload of $n > E$ tokens is delivered by **rendezvous**: the receiver gets the envelope and handle, is charged only the envelope cost, and must explicitly materialise the body.

This is MPI’s eager limit, and it exists for the same reason. Below the limit, pushing bytes unsolicited

	MPI process	LLM agent executor	Consequence for the interface
A1 Determinism	deterministic	nondeterministic; plausible wrong answers	reduction reproducibility becomes an explicitly declared property (Section 5.4)
A2 Scarce resource	memory/network bandwidth	context window occupancy	transfer cost is denominated in tokens; flow control is a context budget (Section 4.2)
A3 Operator cost	nanoseconds; free vs. communication	seconds to minutes; dominates everything	algorithm selection minimises operator applications on the critical path, not messages (Section 5)
A4 Latency distribution	tight	heavy-tailed [9]	strict barriers wait for the maximum of P heavy-tailed samples; hence quorum collectives (Section 5.3)
A5 Failure model	fail-stop, rare, kills the job	individual, frequent, partial, sometimes silent	ULFM adopted wholesale, plus leases, epochs, join deadlines and replacement (Section 6)

Table 1: The five asymmetries between an MPI process and an LLM agent executor.

is cheaper than a handshake; above it, the receiver’s buffer is too precious to fill without permission. Only the identity of the precious resource changes — from unexpected-message memory to the receiver’s attention. The mechanism converts context exhaustion from an emergent failure into a flow-control problem with a tunable threshold. When a charge would exceed the budget the implementation *degrades* — delivering a bounded view with an advisory note — rather than failing, because an agent that receives a truncated message can continue while one that receives an error usually cannot.

Views: the derived-datatype analogue. An MPI derived datatype is a declarative description of *how to access a buffer*: `MPI_Type_vector` says “ten blocks of four doubles, stride one hundred”, and the library gathers exactly those elements. The point is that the access pattern is data the library can optimise rather than control flow it cannot see [42]. `AGENTMPI`’s scarce resource is context, so the analogous question is not *which bytes* but *which tokens, and how few*. A **view** answers it declaratively: `head:800, keys:a,b, lines:120-180, grep:TODO, outline, shape, chunk:2/8`. Views are deterministic and free of model calls, so a replay charges identical context and the runtime can cache them. Semantic compression is deliberately *not* a view: it is an agent-evaluated operation whose cost the harness must see.

Gather returns a manifest. `AMPI_Gather` does not concatenate bodies. It returns one entry per contributor with rank, handle, token count and summary; the caller then materialises what it needs or supplies a per-contribution view budget. Section 8.1 shows why concatenation is not a reasonable default at any interesting P .

4.3 Communicators, labels and topologies

Communicators, groups, `split`, `dup` and Cartesian and graph topologies are adopted essentially unchanged, for the composability reason of Section 2. Two changes matter.

Split waits for live ranks. Because the resulting communicators are immutable, membership cannot be decided from a partial vote; the operation completes when every *live* parent rank has registered a colour, and failed ranks are excluded rather than waited for.

Collectives are identified by label, not program order. MPI matches the k -th collective on a communicator across ranks because the program text says so. An agent’s program order is not reliable — it may retry a command, skip a step, or reorder two independent calls — and relying on order silently mismatches ranks. Named collectives were the single largest robustness improvement we made.

4.4 Windows: shared state

The most-reported failure of real multi-agent systems is that executors cannot share what they learn [7]. Pure message passing forces every fact discovered by one agent to be routed to every agent that will later need it, by an author who cannot know in advance who that is. `AGENTMPI` adopts MPI-3 RMA [25]: named, versioned windows with `Put`, `Get`, `Accumulate`, `Compare_and_swap`, `Fetch_and_op`, active-target `fence` and passive-target locks. Keeping windows named and explicit is what keeps shared state auditable — the difference between a blackboard in the sense of HEARSAY-II [12] and a mess.

Three details are load-bearing. `Accumulate` replaces read-modify-write (three round trips and a race) with one atomic operation, so “union this finding into the shared findings” needs no lock. `Compare_and_swap` is

MPI	AgentMPI
process	rank + <i>epoch</i> (fenced identity)
communicator context	unchanged; nothing in the agent ecosystem has an equivalent
byte	token
eager limit	eager threshold in tokens
unexpected-message buffer	receiver’s context window
rendezvous RTS/CTS	handle + explicit materialisation
derived datatype	view : declarative bounded projection
non-overtaking rule	verbatim
MPI_Op predefined	runtime operator
MPI_Op_create	agent operator + continuation protocol
commutativity flag	default inverted to <i>false</i>
collective by program order	collective by label
algorithm catalogue	unchanged; <i>selection rules rederived</i>
in-network aggregation	journal-mediated flat collectives
MPI_Win, Accumulate, CAS	unchanged
MPI_Win_lock	<i>leased</i> lock with fencing token
MPHX_Comm_revoke/shrink/agree	verbatim (agree gains a quorum)
checkpoint/restart	recovery briefing (durable-execution replay)
blocking forever	deadline + resumable
—	retry context ledger; quorum collectives; join deadline; lease extension

Table 2: Concept correspondence. Bold entries are where the adaptation is substantive rather than cosmetic.

how work is claimed: a task cell holds **unclaimed** and whichever executor swaps it wins, eliminating an entire class of duplicated-work bug, and unlike a lock it cannot be held by a dead executor. And **Win_list** enumerates keys with sizes and writers *without reading bodies*, which is what makes a blackboard usable by an executor with a bounded context: it can see what exists for a few tokens per key and then spend its budget deliberately.

Locks are **leased** and carry a monotone fencing token. An MPI process holding a window lock cannot wander off; an executor can. The lease stops a dead holder wedging the job; the token stops a revived holder corrupting state after expiry.

4.5 Deadlines instead of blocking forever

Every blocking AGENTMPI operation is deadline-bounded, and on expiry leaves its state such that re-issuing the identical operation *resumes the same wait*. MPI_Recv blocks forever, which is tolerable when a peer’s failure kills the job; an agent peer may be slow, wedged

or dead, and no timeout distinguishes them.

Implementations must additionally retry *internally* by default. This is not a convenience. In a pilot run, an executor instructed to retry a timed-out call up to twenty times gave up after two and stalled its entire reduction tree. A protocol that depends on an executor’s persistence is not a protocol; making the default behaviour “keep waiting” means abandoning a collective requires the executor to do something rather than nothing.

5 Algorithms and selection

AGENTMPI keeps MPI’s algorithm catalogue — linear, binomial, chain, ring, recursive doubling, dissemination, plus a journal-mediated **flat** family — and adds one schedule with no MPI analogue. What changes is the selection rule.

5.1 Agent-evaluated operators

Operators come in two families. *Runtime operators* (**concat**, **union**, **jsonmerge**, **sum**, **vote**, **maxby**, ...) are applied by the implementation: exact, deterministic, free. *Agent operators*, written **agent:label**, are applied by an executor. This is MPI_Op_create with a language model as the callback.

An agent operator cannot complete inside one call, because the operator *is* the caller. The reduction therefore returns a **merge directive** naming two operand locations and an output location, charges the caller’s ledger for the operand tokens it will read, and accepts a commit that records the merged value and resumes the schedule. The accumulator and schedule position are checkpointed durably, so a timeout, crash or replacement resumes at the same point. The alternative — having the runtime call a model itself — would make AGENTMPI a framework with opinions about models, credentials and prompting; the continuation structure keeps the library owning the schedule and the user owning the operator, which is exactly MPI’s division.

5.2 The selection rules, rederived

(a) **Runtime operators want no tree.** When the implementation can apply the operator, the shared control plane folds all contributions in place: one round, zero messages. A tree adds rounds and buys nothing. This is the in-network-aggregation regime, and it is why AGENTMPI’s default for a runtime-operator reduction is **flat**.

(b) **Agent operators want MPI’s tree, and reject MPI’s best tree.** When an executor applies the

operator, the serialised operator applications are the entire cost. A binomial tree has depth $\lceil \log_2 P \rceil$ where a linear chain has depth $P - 1$, at identical total work — so here the classical tree matters far *more* than it does for bytes. But recursive-doubling allreduce, MPI’s standard choice for short messages precisely because redundant arithmetic is free [43], performs $P \log P$ applications in total against reduce-then-broadcast’s $P - 1$. Section 8.1 measures the consequence.

(c) Barriers: the shared medium is not free at scale. A journal-mediated barrier counts arrivals in one round, which looks unbeatable. Measured, it beats a dissemination tree only up to $P \approx 32$; beyond that the shared medium becomes the contended resource and the tree wins. AGENTMPI therefore selects **central** for $P \leq 32$ and **dissemination** above — a threshold fitted to measurement, in the spirit of MPICH’s message-size thresholds [43].

5.3 Quorum collectives

A collective may carry a quorum q and a deadline, completing when $\lceil q \cdot \text{live} \rceil$ ranks have contributed. Agent completion times are heavy-tailed (A4), so a strict barrier over P executors waits for the maximum of P heavy-tailed samples [9]; the quorum knob lets the harness choose between bulk-synchronous determinism and bounded staleness, the same trade stale-synchronous parameter servers make [24], expressed as a collective rather than as a training-loop hack.

One detail is easy to get wrong and we did: reaching quorum must *release* a barrier without *closing* it, so a straggler arriving afterwards still passes through. Otherwise a quorum barrier guarantees that precisely the slowest ranks fail.

5.4 Reproducibility

An operator declares whether it commutes; the default is *false*, and a non-commuting operator forces a canonical rank-ordered tree. MPI users are warned that floating-point `MPI_SUM` may differ between runs and process counts because the tree shape varies. For an agent operator the effect is far larger: “merge these two draft glossaries” is neither associative nor commutative nor deterministic, and different tree shapes give materially different results. Inverting MPI’s default is the conservative choice; a harness that knows its operator is order-insensitive can opt into the faster schedule, as in MPI, but with much larger stakes [5].

5.5 A canonical tree gives reproducibility, not consistency

Section 5.4 argues that a non-commuting operator needs a pinned tree shape. That is necessary and it is not sufficient, and E2 showed us why in a form we had not anticipated.

In the eight-rank contract reduction, two branches of the tree independently encountered the *same* conflict — which module defines the shared exception type — and resolved it in *opposite* directions. Both resolutions were carried into the merged result, which therefore contained two contradictory rulings under the same identifier. No merge could have detected this: each one saw a locally consistent pair of operands, and the contradiction existed only in the union. Two ranks then began acting on different halves of the result, and it was caught by a rank reading the published contract afterwards rather than by the collective. Separately, the same reduction dropped four of eight modules from one section of its output despite an explicit instruction never to drop a module, because “never drop” is a global invariant and every merge is local.

Both failures are inherent to reducing with a non-associative operator over a tree, and no choice of tree shape removes them. What a harness can do is structural: carry unresolved conflicts *forward* as unresolved rather than deciding them at an internal node, so the root decides each one exactly once; or verify the closed result against a global invariant. Both are the agent-operator analogue of what reproducible-reduction work does for floating point [5] — change the operator so the schedule cannot change the answer, rather than hoping the schedule is stable. The interface supplies the tree; it cannot supply agreement, and Section 4 now says so.

5.6 A schedule with no MPI analogue

relay broadcast is a binomial tree whose internal nodes *rewrite* the payload: each agent adapts its parent’s message for its own subtree. Bytes do not reinterpret themselves in flight, so MPI has no counterpart; the closest classical relative is a hierarchical reduction run backwards. It is included because it is the schedule a human organisation actually uses to broadcast intent, and because it exposes a failure mode MPI never has — semantic drift accumulating over $\log P$ rewrites.

6 Fault tolerance

We adopt ULFM’s design [4] because its principles are exactly right here: failure notification is local rather than globally consistent; the runtime provides tools and does not choose the recovery strategy; failure-free performance must not suffer. **revoke**, **shrink** and **agree**

are taken essentially verbatim, including the observation that makes **revoke** indispensable — when a rank fails, the *survivors* are the problem, because each is blocked inside a collective that can never complete and will only notice if it happens to be waiting on the dead rank directly.

Four additions are forced by A5.

Two-phase detection. A timeout cannot distinguish a thinking executor from a dead one [8, 14], and the two errors are not symmetric: declaring a working agent dead discards minutes of real work and then fences it. Detection is therefore two-phase, in the spirit of accrual detectors [23]. Phase one marks a rank **suspect** when its lease expires: visible to all, but nothing changes — collectives keep waiting and no work is redistributed. Phase two declares **failed** only after suspicion persists for a confirmation window. Evidence of activity clears suspicion. The launcher may convict immediately, which is the right authority, since unlike the protocol it knows whether the executor process exists. We arrived at this after a single-phase detector killed a translator agent that had been doing 580 seconds of legitimate work.

Join deadlines. A rank whose executor never starts has no lease to expire, so it is neither alive nor failed and every peer waits forever. We hit exactly this: a launcher that could start only 6 of 22 requested ranks produced a job with 16 permanently pending no-shows and no operation able to detect it. Granting the lease when the rank is *requested* rather than when it first calls in makes launch failure an ordinary detectable failure.

Lease extension. Because the lease also bounds how long a blocked peer waits, making it longer than the worst legitimate pause is the wrong fix. Instead the executor may declare what a timeout cannot infer: `ampi hb -extend 900` — “I am about to spend a long time on one step” — the same move a long-running task makes when it extends a distributed lock rather than acquiring a worst-case one. Section 9 reports that executors do this unreliably, which is why two-phase detection is necessary rather than merely nice.

Replacement and the recovery briefing. There is no memory image to restore, so process checkpointing [11] has no analogue. What does have one is durable execution: replay the record of externally visible commitments. `AMPI_Recover` answers the five questions a replacement must know and cannot guess — what was I assigned, what did I publish, what did I receive, what did I promise that is outstanding, and what did I record for myself — from the journal, the window history and a durable memo table the executor writes deliberately.

Supervision bounds replacements per rank, following OTP’s max-restart-intensity [3]: an executor that fails because its assignment is impossible will fail again.

Finally, a blocked rank has three obligations while waiting: renew its own lease, run the detector, and observe revocation. Omitting the first is catastrophic and not obvious — see Section 9.

7 Implementation

The reference runtime is a dependency-free Python library plus an `ampi` command-line binding, about 10 000 lines, with a 714-line conformance suite of 50 tests that drive the CLI in subprocesses exactly as an agent would. All protocol state is durable: a per-job SQLite journal in WAL mode holds the matching queues, communicator table, request table, window contents, collective state and event trace, and a content-addressed filesystem store holds payload bodies. Durability is not a feature bolted on for recovery; it is what makes a replacement executor possible at all, and it makes tracing free.

Choosing an embedded store rather than a daemon is deliberate: atomic multi-table transactions are needed because message *matching* must atomically pop from one queue and push to another, and a daemon would be a single point of failure the design would then have to defend. Blocking operations poll, with a three-phase cadence (fast spin, then 20 ms, then capped at 0.5 s) so that the runtime’s own latency stays out of measurements while a rank waiting ten minutes costs nothing.

The binding is a CLI. For an LLM executor the binding is a command-line tool, because that is the only interface an agent reliably has to a stateful library: it cannot hold a handle across turns, cannot link a shared object, and its function calls are invocations whose output lands in its context window. Designing the binding mattered as much as designing the protocol. Figure 1 shows a rank participating in an agent-evaluated reduction.

Four binding properties earn their complexity: ambient identity; terse structured output with sizes in tokens and an explicit next-action line; errors that prescribe (every error carries a hint naming the command to run next, and retryable errors say so); and idempotence by default — labelled collectives, idempotency keys on sends, resumable blocking calls. Errors are read by language models, and one that says what to *do* is acted on where one that says only what went wrong often is not.

Tracing and a viewer. Every job records an event trace with the vocabulary the HPC tools use — per-rank intervals, matched send/receive arrows, collective

```

$ ampi allreduce --op agent:reconcile_glossary \
  --label glossary --in @terms.json \
  --algo reduce_bcast --timeout 120
AMPI_SUCCESS coll=k:8f1c complete=false
action_required=merge step=s:a91e round=1
context_charged=812

REDUCTION STEP (round 1, operator
'reconcile_glossary'). Combine the two operands
into ONE result of the SAME shape:
  left (yours, rank 4): .../r4_round1_left.txt
  right (from rank 6): .../r4_round1_right.txt
Write the combined result to .../r4_round1_merged.txt
then run:
  ampi reduce-commit --step s:a91e --in @<that file>

$ ampi reduce-commit --step s:a91e --in @merged.json
AMPI_SUCCESS coll=k:8f1c complete=false
action_required=merge step=s:c02d round=2
...
$ ampi reduce-commit --step s:f77b --in @merged.json
AMPI_SUCCESS coll=k:8f1c complete=true handle=o:2b9d
tokens=1194 merges=3
----- payload -----
{"Flatland": "...", "Lineland": "...", ...}
----- end payload -----

```

Figure 1: An executor applying an agent operator at two internal nodes of a binomial reduction tree. The runtime owns the schedule; the agent owns the operator. Charging the context ledger for the operand files it is told to read keeps the accounting honest, since it certainly will read them.

intervals with participant join/done times, window accesses, failures and error events. We also ship a timeline viewer in the lineage of Jumpshot and Vampir, because a multi-agent run otherwise produces no artefact a human can read: the per-agent transcripts are unordered, and the questions one actually asks (“who was waiting on whom when the job stalled”, “whose context filled first”, “did the reduction tree have the shape I asked for”) are questions about a timeline.

8 Evaluation

We separate two kinds of measurement, as HPC papers do. Section 8.1 uses *stub executors* — threads calling the runtime directly, with their own journal connections and an optional simulated think time — to isolate cost terms and check the selection rules at P up to 128. Every number there is a statement about the *runtime*, not about agent behaviour, and we label it as such. Section 8.2 reports multi-agent runs in which every rank is a separate LLM agent coordinating only through the protocol.

8.1 Microbenchmarks

Cost model parameters. Ping-pong regression over 25 repetitions at payload sizes from 8 to 32 768 tokens (Figure 2) gives $\alpha = 5.7$ ms and $\beta = 1.22$ μ s/token, so $n_{1/2} = 4720$ tokens. Token counts use tiktoken:cl100k_base. The practical reading is that the runtime contributes milliseconds where γ and λ contribute

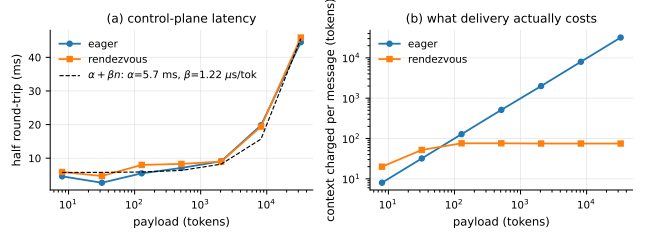


Figure 2: Control-plane latency and context cost versus payload size. The eager/rendezvous crossover is visible in (b): rendezvous charges the receiver an envelope regardless of payload size.

seconds to minutes: the protocol is not the bottleneck, and optimising it further would be misdirected effort.

Figure 2(b) is the cleanest single demonstration of the flow-control mechanism. Under eager delivery the context charged per message tracks the payload exactly, from 8 tokens to 31 886. Under rendezvous it is *flat at about 75 tokens across the entire range* — the envelope — because the body never enters the receiver’s context unless the receiver asks for it. That flatness is what makes the collective results in Table 3 possible.

Collective volume confirms the analysis. Figure 3 plots messages and payload tokens against P . Ring allgather performs $P(P - 1)$ messages (16,256 at $P = 128$) and moves 20,807,680 tokens against recursive doubling’s 896 messages and 325,120 tokens, a factor of 64.0. Dissemination barrier performs $P[\log_2 P]$ messages and linear $2(P - 1)$, both exactly as predicted. The *flat* family performs zero messages at every P , which is the control-plane observation of Section 3 made concrete.

MPI’s short-message optimum is the wrong choice for agent operators. At $P = 128$, recursive-doubling allreduce performs 896 messages moving 3,266,560 payload tokens, against reduce-then-broadcast’s 127 messages and 89,921 tokens: **36.3 $\times$ the data volume**. In MPI that extra volume is worth paying because it removes a broadcast phase and the redundant arithmetic is free. Under Equation (1) the arithmetic is not free and the volume is charged to context, so the ranking inverts. This is the paper’s clearest demonstration that the catalogue transfers but the selection rule must be rederived.

Agent-operator reduction depth. With a simulated $\gamma = 0.25$ s (Figure 4), a binomial tree at $P = 64$ gives the busiest executor 6 merges — matching $\lceil \log_2 P \rceil$ exactly at all 6 measured points — and completes in 2.55 s, against the linear chain’s 29.67 s: a speedup of 11.6 $\times$ at identical total work (63 applications either way). The chain’s serialised depth is

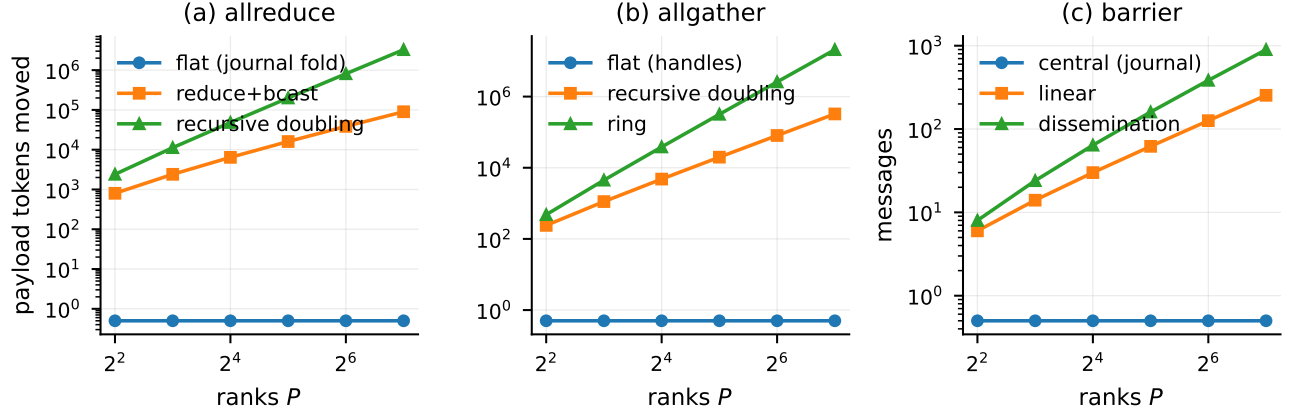


Figure 3: Collective cost versus P . Journal-mediated (**flat**) schedules move no messages at all; the tree and ring schedules reproduce their textbook message and volume complexities.

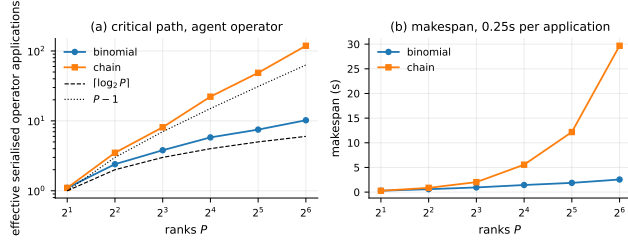


Figure 4: Agent-evaluated reduction. (a) effective serialised operator applications; (b) makespan at $\gamma = 0.25$ s. Total work is identical; only the dependency depth differs.

$P - 1 = 63$ even though each rank performs one merge, which is exactly the distinction between per-rank work and dependency depth.

Context cost is where the protocol earns its keep. Figure 5 and Table 3 report tokens delivered into context windows at $P = 128$ with 4000-token payloads. An allgather that inlines bodies charges one rank 501,888 tokens — far beyond any context window — and 64,241,664 across the job. Handle-based delivery charges that rank 3200 and 409,600 in total: $157 \times$ and $157 \times$ reductions respectively. Broadcast shows the same effect at smaller magnitude ($501,888 \rightarrow 5120$). Bounded views sit between the two, trading fidelity for a fixed per-contribution cost. This is the concrete mechanism behind “context exhaustion” in agent harnesses, and it is a flow-control choice rather than an inevitability.

Matching does not degrade with queue depth. Receive latency is flat from an empty unexpected queue (2.80 ms) to 2048 queued messages (0.60 ms). MPI implementations pay a linear scan of the match list, and several studies measure the cost; an indexed durable match list does not, which is a concrete benefit of

Collective	Delivery	Total tokens	Peak in one rank
bcast	inline	501,888	3921
	handle	5120	40
	view (400)	51,200	400
allgather	inline	64,241,664	501,888
	handle	409,600	3200
	view (400)	983,040	7680

Table 3: Tokens entering agent context windows, $P = 128$, 4000-token payloads.

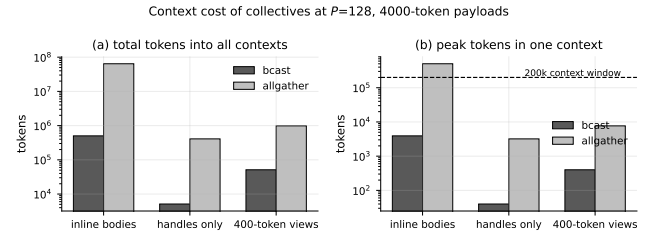


Figure 5: Context cost by delivery discipline. The dashed line marks a 200 k-token context window: inlined allgather exceeds it by more than $2 \times$ at $P = 128$.

the journal-as-network choice rather than an assertion about it.

Barrier crossover. Figure 6 shows the journal-mediated barrier beating a dissemination tree at $P = 32$ (0.67 s vs. 1.08 s) and losing at $P = 128$ (9.16 s vs. 6.28 s); the crossover lies between 32 and 64, which is the measurement behind the selection threshold in Section 5.2. The shared medium behaves like a switch with in-network aggregation up to a point and like a contended resource past it — the same reason MPI implementations abandon linear collectives at scale.

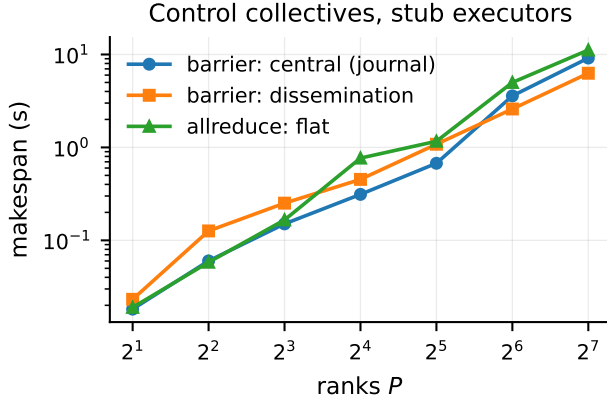


Figure 6: Control collectives with stub executors. The shared medium is not free at scale.

8.2 Multi-agent applications

Each rank below is a separate LLM agent with its own context window, launched with a prompt containing its identity, its assignment and the protocol manual. Agents coordinate *only* through `ampi`; there is no shared conversation and no orchestrator reading their output.

Arms are ablations, not strawmen. Every application runs twice. The *naive* arm has the same agents, the same task, the same assignment and the same shared filesystem, and still uses AGENTMPI for identity, launch and metering — it simply uses it the way a harness written without the protocol’s discipline does: no negotiated agreement, no boundary exchange, and a final gather that materialises full bodies. Comparing against a baseline that also lacked instrumentation would produce numbers we could not interpret.

8.2.1 E1: parallel book translation

Translation looks embarrassingly parallel, and a naive harness scales trivially. What makes it a real parallel programming problem is that the pieces are not independent: two translators who never talk render invented terminology differently, and section $i + 1$ does not follow from how section i ended. The dependencies are a *global agreement* (terminology) and a *nearest-neighbour agreement* (boundaries) — precisely the two dependency structures MPI exists to express, namely a reduction and a halo exchange.

Corpus: Abbott’s *Flatland* (1884, public domain), which is unusually dense in invented vocabulary that must be rendered consistently (Flatland, Lineland, Regularity, Chief Circle, the Colour Bill). Target: Simplified Chinese. Each rank owns one section. The AGENTMPI arm performs (i) `AMPI_Scatter` of assignments, (ii) `AMPI_Allreduce` with an *agent* operator to reconcile per-section terminology proposals into one glos-

sary, (iii) translation using that glossary, (iv) a halo exchange over a 1-D Cartesian topology in which each rank sends its closing sentences downstream and revises its opening, and (v) handle-based `AMPI_Gather`. The *naive* arm omits (ii) and (iv) and materialises bodies in (v).

Metric. For each probe term we take the rendering each rank reports having actually used and compute the share held by the modal rendering among ranks whose section contains that term. A term rendered identically everywhere scores 1.0; three renderings among three ranks scores 0.33. Glossary adherence is checked against the *collective’s own result object* in the journal, not against anything an agent claimed, because a protocol that agrees a value nobody then uses has bought nothing. Boundary revision is read from window cell versions, which is durable evidence rather than self-report.

Results. Table 4 reports both arms at $P = 5$. Both completed every section, in statistically indistinguishable wall time (833 s vs. 833 s) and with comparable output volume, so the protocol arm’s coordination cost did not show up as latency. What differs is the artefact and the context.

Of 10 probe terms reported by two or more ranks, the unprotocoll arm diverged on two — *Equilateral* was rendered two ways in a two-two split, and *Priest* two ways by two ranks — giving a mean modal share of 0.900 and 0.800 of terms fully consistent. The AGENTMPI arm diverged on none: mean modal share 1.000, all terms consistent. The agreed glossary contained 14 terms, produced by 4 agent-evaluated merges with 3 on the critical path — matching $\lceil \log_2 P \rceil$ — and adherence to it, checked against the collective’s own result object rather than against agent self-report, was 1.000. The halo exchange caused 4 of 5 ranks to revise and republish their opening (a window-cell version above one), against none in the unprotocoll arm.

Peak context in a single rank was 8661 tokens against 897 — a factor of 9.7 — because the unprotocoll arm’s coordinator materialised every translation while the AGENTMPI arm’s collected handles.

How much to conclude. This is one run per arm at $P = 5$ with 10 scored terms, so the terminology effect rests on two divergences. We report it as a demonstration that the mechanism does what it is supposed to do — the glossary was agreed, adherence was total, and the divergences the unprotocoll arm exhibited are exactly the ones a glossary prevents — and not as an effect size. The context result is not sample-limited: it is a structural consequence of handle-based collection and matches Table 3.

8.2.2 E2: collaborative software development

Software development is the tightly-coupled case, and it is hard in the way tightly-coupled parallel codes are

	no protocol	AGENTMPI
ranks	5	5
sections completed	1.00	1.00
mean modal share	0.900	1.000
fully consistent terms	0.800	1.000
distinct renderings/term	1.20	1.00
peak context (tokens)	8661	897
total context (tokens)	8661	2525
timeout retries	0	0

Table 4: E1, parallel book translation. Glossary adherence in the AGENTMPI arm was 1.000 over 14 agreed terms, produced by 4 agent-evaluated merges with 3 on the critical path.

hard: the pieces depend on each other mutually, the interfaces between them are not given in advance, and a disagreement about a shared representation does not degrade the result — it produces something that does not run.

Eight agents implement MiniScheme, a Scheme interpreter in Python: reader, value representation and printer, environments and closures, special forms, evaluator with proper tail calls, and three primitive modules, plus an integrator. The *public* behaviour is fixed by a specification; the *internal* interfaces — how a pair is represented, what an environment lookup returns for an unbound name, how a special form hands a tail expression back to the evaluator — are deliberately unspecified. Agreeing them is the coordination problem. Grading is by a 174-case held-out suite the agents never see, run per-case in a fresh subprocess with a wall-clock limit, so a module that loops forever costs one test rather than the run.

The AGENTMPI arm reduces eight interface proposals into one contract with an agent operator, publishes it in a window, keeps an append-only decisions log written with lock-free `accumulate`, takes a leased lock around the single genuinely shared file, and runs two synchronised integration rounds separated by `Win_fence` and `Barrier` with test failures distributed by `AMPI_Bcast`. The *naive* arm has the same eight agents, the same specification and module assignment, and a shared filesystem they may read freely — a strong baseline, since a shared repository with no coordination protocol is how most multi-agent coding harnesses actually work.

Results: a null result, and a more interesting one behind it. Both arms passed **174 of 174** held-out cases — every case, in every category, in both arms — having also each passed all 43 visible smoke tests. The negotiated contract made no difference to the outcome. We report that plainly because it is what happened, and then we report what the journals show about *why*, which we think is the more useful finding.

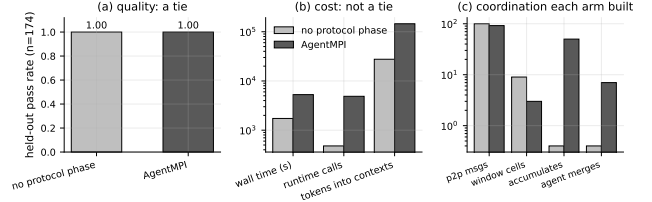


Figure 7: E2. Both arms passed every held-out case, so (a) carries no information; (b) and (c) do. The unprotocoll arm created more window cells and sent more point-to-point messages than the protocol arm — it built its own coordination layer — while the protocol arm’s prescribed contract phase accounts for almost all of its extra cost.

The unprotocoll arm rebuilt the protocol, on its own, in ninety seconds. The *naive* arm’s program gave it no coordination phase: no contract, no decisions log, no synchronised rounds. What its agents did instead is the most striking thing in this paper, and the journal timestamps it precisely.

Beginning at $t = 13.3$ minutes — before most modules existed — 7 of the eight ranks each sent an identical interface-declaration message to *every* peer, within about ninety seconds of one another. That is an allgather, hand-rolled, unprompted, by seven agents independently. The integrator then wrote an `interface` cell into a shared window at $t = 14.9$ and a `datum-expectations` cell at $t = 16.3$; that is a published contract. It broadcast `integration-status` at $t = 27.7$ and again at $t = 33.2$; those are integration rounds. In total the arm performed 18 all-to-all broadcasts and created 12 window cells, against the protocol arm’s 3 — because the protocol arm had `AMPI_Bcast` and `AMPI_Allreduce` and did not need to build them.

We find this far more persuasive than a pass-rate difference. A pass-rate difference shows one harness beating another. Seven of eight agents independently converging on “broadcast what you provide to everyone, publish the agreement into shared state, broadcast integration status” — which is an allgather, a window put and a broadcast — is evidence that the *abstraction* matches the shape of the problem rather than the harness matching the task. It is also the sharpest possible caution against overselling: these mechanisms are not the protocol’s contribution. Agents reach for them when they are absent. What the protocol contributes is that they can be named, scheduled, bounded and read back afterwards.

And then it invented something the protocol does not have. The ablated arm did not only reconstruct our phases; it adopted a different coordination strategy for the part it could not schedule. Rather than agree an interface in advance, each consumer *discovered* its producer’s actual interface at runtime. The evalu-

ator probes every special-form handler with synthetic forms at first evaluation and settles the calling convention by majority vote over the probe results instead of assuming it. Handlers resolve the compound procedure class by searching two peers’ modules for any of several plausible names, and validate a candidate environment frame by binding a probe into it and checking visibility and parent linkage. The shipped source carries 54 probe- and detection-related identifiers against the protocol arm’s 10, and 10,391 lines against 5394 for the same graded behaviour.

This is a real alternative to agreement-by-reduction, with a real advantage. Empirical discovery *cannot* produce the contradiction of Section 5.5: nobody adjudicates the same conflict twice in opposite directions, because nobody adjudicates it at all — each consumer simply observes what its producer does. Against that, every consumer carries adapters and fallbacks, roughly doubling the code, and the detection logic itself becomes a defect surface. It did: two ranks introduced bugs in their own detection, and one of them lived in the *interaction* between two ranks’ detectors — a convention latched last-write-wins during a peer’s two-convention probe — which reproduced only for the first special form evaluated in a fresh process, and which no visible test caught.

So the two arms are not “coordinated” and “uncoordinated”. They are two coordination strategies: a priori agreement, which risks deciding the same question inconsistently in different places and therefore needs a scheduled point where it is decided once; and runtime discovery, which cannot be inconsistent but pays for it in adapters and in bugs inside the adapters. Section 10 argues that the protocol should offer the second as well as the first.

The prescribed process was expensive. Table 5 and Figure 7 give the cost. The AGENTMPI arm took 5280 s against 1732 s ($3\times$), made 4882 runtime calls against 479, and delivered 145,243 tokens into agent contexts against 27,623 ($5.3\times$). Almost all of that is the phase our harness imposed: agree a complete contract by agent-evaluated reduction *before* anyone writes code, then two synchronised integration rounds. On this task that was wasted work.

The honest reading is that this is a harness design error, not a protocol result. The *naive* arm used the same primitives opportunistically — publish an interface when you have one, read a peer’s when you need it — and got the same artefact for a third of the wall time. A harness author with these primitives should prefer that, and reach for the full contract reduction only when opportunistic agreement demonstrably fails.

	no protocol	AGENTMPI
held-out cases passed	174	174
pass rate	1.000	1.000
differential defects	1	0
wall time (s)	1732	5280
runtime calls	479	4882
tokens into contexts	27,623	145,243
point-to-point msgs	100	92
window cells created	9	3
lock-free accumulates	0	50
agent-evaluated merges	0	7
interpreter lines	10,391	5394

Table 5: E2, eight ranks per arm, graded by a 174-case held-out suite. The quality outcome is identical; the cost is not. The unprotocolled arm created its own contract and decisions windows without being asked to.

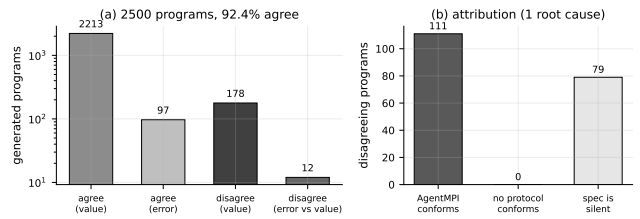


Figure 8: Differential testing over 2500 generated programs, where the held-out suite had tied. The asymmetry in (b) is the result; the parenthesis in its title is the honesty — all 111 attributable observations trace to one root cause.

A discriminating instrument: differential testing. A saturated suite bounds each arm from below and measures neither, and extending it by hand after reading one arm’s failures would not be an experiment. Differential testing gives a way out that is symmetric between the arms by construction: generate programs from a grammar written against the specification, run each under both interpreters, and compare. Every disagreement is a defect in at least one arm, and no program is chosen because we already knew it would fail. The generator is seeded, so the run is reproducible.

Over 2500 generated programs (seed 11) the two arms agreed on 92.4 % — 2213 identical values and 97 cases where both correctly raised an error — and disagreed on 190. Attribution is deliberately conservative: for numeric formatting the referee is Python’s own `repr`, not our judgement, and otherwise we attribute only where the specification settles the question and we quote the clause. On that basis **111 of the 190 disagreements are attributable, and every one of them favours the AgentMPI arm**; none favours the ablated arm. The remaining 79 are cases the specification does not settle, chiefly the value of a degenerate (`case x`) with no clauses, and we attribute nothing to them.

One defect, not a hundred and eleven. Program-level counts overstate defect counts badly: one wrong printer appears in every generated program that happens to

print a float. Fingerprinting the differences shows 1 attributable root cause behind all 111 observations — float exponent formatting, where the ablated arm renders `1e21` and `1e-7` against Python’s `1e+21` and `1e-07`, contrary to the specification’s requirement that floats print “Python `repr`-like”. The honest claim is therefore narrow: differential testing found *one* specification violation in the ablated arm and none in the protocol arm, in a region the held-out suite did not probe.

Why that one defect is the interesting one. It is interesting because the same defect class arose in *both* arms, and only one arm’s process removed it. The journals record both halves.

In the AGENTMPI arm, the integrator raised it during integration round 2 — “floats print as `1e20` where Python `repr` gives `1e+20`” — recorded it in the shared fix log, a third rank independently seconded the recommendation in the same log, and the printer’s owner adopted it. In the ablated arm, a rank diagnosed the same problem more precisely than anyone in the protocol arm did, messaged both the owning rank and the integrator twice with a one-line fix and a second data point (“(`number->string 1e21`) $\rightarrow$ “`1e+21`” but (`write 1e21`) $\rightarrow$ `1e21` ... so `number->string` disagrees with the printer for any float that `reprs` with an exponent”), fixed its own side by delegating, and was never answered. The defect shipped.

So the mechanism that mattered was not the negotiated contract, which the ablated arm reinvented anyway. It was the *synchronised integration round*: a scheduled point at which the integrator broadcasts failures, assigns them, and everyone records what they changed in one place. The ablated arm had peer-to-peer messages and used them well; what it lacked was a place where “what does the specification say” got adjudicated once and written down. That is a claim about one defect in one run, and we state it as such — but it is a mechanism-level claim, arrived at by an instrument that could not have favoured either arm, and it is the kind of thing a pass-rate difference would not have told us.

Methodological disclosure. The generator’s coverage was extended once. The first version reached perfect agreement, and both teams’ own agents had reported residual defects, so we knew it was not reaching the relevant regions. We therefore added whole grammar productions — degenerate special forms, floats that print in exponent notation, integers past CPython’s digit-conversion limit — and not the specific failing instances the agents named. The extension is symmetric between arms and was made before any per-arm result was computed from it. A reader who wants the pre-extension result has it: perfect agreement, and no information.

Mechanism	Count	Reading
agent merges (contract)	7	$P - 1$, with 3 on rank 0: the binomial schedule exactly
contract size	21 638 tok	published once as a window cell, not broadcast
decisions-log writes	31	lock-free accumulate: no rank serialised on a lock to record a decision
fix-log writes	19	two synchronised integration rounds
lock acquisitions	8	the one genuinely shared source file
timeout retries	46	every one resumed the same wait; none abandoned
peak context per rank	12.1–22.1k tok	against a 140k budget

Table 6: E2 mechanism evidence under real agent load, from the journal.

Why the held-out suite did not discriminate.

Three conditions made agreement easy, and each identifies where we would expect the result to change. The agents *shared a filesystem*, so a rank could read its dependency’s source instead of asking about it — remove that and the negotiated contract becomes the only channel. The public specification *pinned the observable behaviour tightly*, leaving the internal representations underdetermined but strongly suggested. And the artefact was a Scheme interpreter, about which current models hold strong shared priors: a `Pair` class with mutable fields, primitives in a dict, a trampoline for tail calls. A task with genuinely novel internal interfaces, or eight ranks that cannot see each other’s files, is where we would expect the contract to earn its cost. We did not run that experiment.

What the run does establish. Table 6 audits the mechanisms under real agent load, which is worth having independently of the comparison. The reduction tree’s shape is the cleanest check: rank 0 performed exactly $\lceil \log_2 8 \rceil = 3$ merges, rank 4 performed 2, ranks 2 and 6 one each, and no other rank performed any — the binomial schedule, $P - 1 = 7$ applications in total, executed by eight independent agents over 46 minutes with no shared context. The contract it produced was 21 638 tokens, published once as a window cell; inlining it to eight ranks would have cost 173 104 tokens of context against the 21 638 one handle-based publication costs.

The 46 timed-out calls deserve a sentence. Every one was a rank waiting for a peer that was still writing code, and every one resumed rather than restarted, because the internal-retry default of Section 4.5 made persistence the runtime’s responsibility. In the pilot run that

Mechanism	Evidence from the journal
Join deadline (Section 6)	16 ranks that never called <code>AMPI_Init</code> were detected and declared failed. Without a lease granted at request time they would have been neither alive nor failed, and every collective would have waited forever.
Fault-tolerant reduction tree	6 dead subtrees dropped; the agent-evaluated glossary allreduce closed on the survivors' contributions after 3 agent merges.
Local failure notification	4 <code>AMPI_ERR_PROC_FAILED</code> returns, each raised at the rank that happened to be waiting on a dead peer — not broadcast.
Deadline-bounded blocking	6 <code>AMPI_ERR_TIMEOUT</code> returns, all resumed by re-issuing the identical call.
Fencing (Section 4)	1 <code>AMPI_ERR_FENCED</code> : a rank declared failed after a long silent step was stopped rather than allowed to keep publishing into a job that had written it off.
Quorum collectives	the final barrier released on the survivors and closed in 4.2 s.

Table 7: E3 audit. Every row is read from the journal, not from an agent's report.

motivated that default, the same situation ended with an executor abandoning a collective after two timeouts.

8.2.3 E3: mass launch failure

Our agent host enforces a limit of ten concurrent background agents. A run requested at $P = 22$ therefore started only 6 ranks, and the remaining 16 never called `AMPI_Init`. Rather than discard it we let it proceed, because a 73 % launch failure exercises the whole of Section 6 at once and does so with none of the artificiality of injected faults: nobody chose which ranks would fail, when, or in what state.

The run **completed**. All 6 surviving ranks finalised, and 5 of 5 collectives closed — the assignment scatter, the agent-evaluated glossary allreduce, the result gather, a numeric allreduce and the final barrier — with 16 of 22 ranks declared failed. The survivors produced 6 complete translated sections in the shared window, in 2411 s. Table 7 audits which mechanism fired.

The single `AMPI_ERR_FENCED` is the episode that motivated two-phase detection (Section 6). One surviving

translator went silent for 580 s of legitimate work — it had been instructed to extend its lease before long steps and did not — and the single-phase detector in use at the time convicted it. Conviction is terminal by design, so the rank could not finish what it had. Under the two-phase detector now specified, that silence would have raised suspicion without redistributing its work, and its next call would have cleared it.

What this run does and does not show. It is a demonstration of graceful degradation, not a measurement of it: we did not choose the failure pattern, so we cannot report recovery cost as a function of failure rate. What it does establish is that the mechanisms compose — join deadlines, local notification, subtree dropping, fencing and quorum release all fired in one run, unrehearsed, and the job produced a coherent partial artefact instead of hanging or aborting. For a first evaluation of a fault-tolerance design that is the property we most wanted to check, because the usual failure of such designs is that each primitive works in isolation and they deadlock together.

8.3 Status of the experimental results

Every number in this paper is generated from a results file rather than typed, and a claim whose measurement is absent fails the build instead of going stale. At the time this draft was generated:

- **Complete:** all microbenchmarks (Section 8.1); E1 both arms at $P = 5$; E2 both arms at $P = 8$, each graded against the full 174-case held-out suite and cross-compared by differential testing over 2500 generated programs; the E3 fault-tolerance audit.
- **Not attempted:** application runs above $P = 8$; a task with genuinely novel internal interfaces or without a shared filesystem, which Section 8.2.2 argues is where the E2 comparison would discriminate.

Section 8.4 discusses the threats to validity, of which the most serious is identity drift in the agent host.

The harnesses, the grading suite, the probe-term list and the analysis scripts are all in the artefact, and `scripts/reproduce.sh` regenerates everything that does not require an agent host and prepares the launch plans for what does.

8.4 Threats to validity

Identity drift contaminated every run. As Table 8 records, the host's shared shell sessions caused stray `AMPI_Init` calls in four of five runs. We report the affected results anyway, for three reasons, and readers should weigh them. First, the contamination is *auditable*, and we audited it: every window write records its writer's rank and epoch, and in the E1 protocol arm all ten scored cells (five drafts, five reports) carry exactly the rank the assignment intended, with zero mis-

matches. In E2 all ten declared module files are present and none is undeclared, so per-file ownership held. The one weaker case is the E1 ablated arm, which writes files rather than window cells, so its attribution rests on per-rank working directories rather than on the journal. Second, re-initialising a rank that is already *running* is treated as a heartbeat rather than a new epoch, so a stray `init` did not displace its victim; the agents detected the drift, pinned their identity inline, and completed their assigned work. The exception is E3, where drift and genuine launch failure are partly confounded: some ranks that appear failed had agents running under another rank’s identity for part of the run. We therefore present E3 as a demonstration that the mechanisms compose, not as a measurement of failure rate, which is how Section 8.2.3 already framed it.

Single runs. Every application result is one run per arm. Agent runs are expensive and our concurrency ceiling was ten, so we have no variance estimates. Where an effect is small we decline to claim it; where it is structural — context cost, schedule shape, message counts — it does not depend on sampling.

Application scale. The concurrency ceiling bounded real-agent runs at $P = 8$, well below the microbenchmark range. The selection thresholds in Section 5.2 are validated with stub executors, not agents.

Our harnesses had bugs, and they are in the results. Three of the fifteen lessons in Section 9 are defects in the experiment harnesses rather than in the protocol: a conditional send against an unconditional receive (L14), a round-two tag mismatch, and a prescribed schedule far heavier than the task needed (L13). These cost wall time and one operator intervention, all disclosed. We report them because a paper that presented only the runs that went smoothly would be describing a different protocol from the one we built.

9 What only real agents exposed

Several protocol defects were invisible to a unit test suite and to stub executors, and appeared the first time real agents ran the protocol. We report them because they are the paper’s most transferable content.

L1. Executors give up far earlier than instructed. Told to retry a timed-out call up to twenty times, a rank stopped after two and stalled its whole reduction tree. The fix is not a better prompt: blocking operations now retry internally by default, so persistence is the runtime’s responsibility and abandoning a collective requires the executor to do something rather than nothing.

L2. Blocking is not evidence of death. An early runtime had ranks make no calls while waiting inside a collective, so the lease-based detector declared each rank that arrived *first* to be dead, and the job cascaded — a 12-rank demonstration took 94s with every barrier timing out. Making the progress engine renew the caller’s lease, run the detector and observe revocation took the same run to 8.9s with all survivors recovering. A blocked rank has obligations.

L3. Executors do not volunteer liveness. Given an explicit lease extension primitive and an explicit instruction to call it before long steps, an agent did not, and was declared dead mid-translation. Since being declared dead is terminal, the detector must not depend on cooperation: hence two-phase detection, and hence the launcher — which actually knows whether a process exists — as the authority for immediate conviction.

L4. Program order is not a reliable collective identifier. Agents retry commands, skip steps and reorder independent calls. Identifying a collective by position silently mismatches ranks. Explicit labels were the single largest robustness improvement we made.

L5. A rank that never starts must still be detectable. See Section 8.2.3: without a lease granted at request time, a no-show is neither alive nor failed.

L6. Quorum must release without closing. Closing a barrier on reaching quorum guarantees that precisely the slowest ranks fail. Straggler-tolerant synchronisation has to let latecomers through.

L7. Errors must prescribe. Agents act on a hint naming the next command far more reliably than they infer a remedy from a description of the fault. This is why every AGENTMPI error carries a hint and why retryable errors say so in their own output.

L8. Ambient identity without an assertion is unsafe, and we had to be shown. An earlier draft of Section 4 argued that making rank ambient “eliminates the entire class” of wrong-rank errors. It eliminated the error we designed against — agents *passing* the wrong rank — and replaced it with a worse one: agents silently *being* the wrong rank. Our agent host shares shell sessions between concurrently running agents, so `AMPI_RANK` was overwritten between one call and the next. The journals record the consequence exactly: in each of four independent runs, one specific rank accumulated *eight or nine* `AMPI_Init` events, one from nearly every other agent in the job (Table 8). Every agent’s first `init` ran as somebody else. In one run it escalated: the victim’s

Run	<i>P</i>	stray <code>init</code> events
E1 (launch failure)	22	8 on rank 2
E1b, protocol arm	5	none
E1b, ablated arm	5	9 on rank 4, 2 on rank 3
E2, protocol arm	8	8 on rank 1
E2, ablated arm	8	8 on rank 7

Table 8: Identity drift, from the journals. One rank per run absorbed a stray `AMPI_Init` from nearly every other agent, because the host’s shell sessions were shared and one agent’s exported `AMPI_RANK` leaked into the others. Because window writes and messages record their writer, the contamination is auditable rather than merely suspected.

own calls were credited elsewhere, its lease starved, and it was declared failed and fenced while actively working.

Nineteen of twenty-two agents reported this independently, and every one reached the same workaround — prefix the identity inline on every invocation — because the interface offered nothing better. It now does: `-expect-rank` and `-expect-job` fail with `AMPI_ERR_IDENTITY` before the operation runs, an explicit root argument overrides the environment variable, every command echoes the identity it acted as, and the launcher issues a per-rank token that the runtime checks — so a drifted rank is told which rank it actually holds credentials for. Ambient identity remains the default, because an agent forced to thread its rank through every call will get *that* wrong instead. What changed is that asserting it is now possible and cheap.

Two design decisions limited the damage and are worth requiring of any implementation. Re-initialising a rank that is already *running* is treated as a heartbeat rather than a new epoch, so a stray `init` did not fence its victim. And every window write and message records its writer’s rank and epoch, so the corruption was auditable afterwards — which is how we can quantify it at all rather than merely suspecting it.

L9. Never truncate an operand. A reduction operand is the input to a function, so removing part of it does not degrade the result, it corrupts it — and for structured payloads it is worse, because clipping JSON mid-string yields a document the operator cannot parse. We shipped the naive behaviour and it bit immediately: agents serving as internal nodes received operands cut mid-string with an elision marker spliced in, and independently invented recovery hacks, one prefix-matching the content-addressed object store by hand, another reordering its output so the decisions it could not afford to lose came before the part clipping would take. The asymmetry to respect is that a *result* may be summarised, because its consumer is a reader; an *operand* may not, because its consumer is a function.

L10. Print only commands that exist. Our reduction directive told the agent to run a subcommand spelled with a space where the real one is hyphenated. Roughly ten agents reported it, several while peers were blocked behind them. A binding that prints a command an agent cannot copy-paste is worse than one that prints nothing, because the agent trusts it.

L11. Give every payload a free path to disk. Agents that wanted to keep a large result without paying context for it reached into the object store directly, or paid. Both are our fault: every operation that hands back a payload needs an option to write it to disk, since nothing enters a context window that way.

L12. When a protocol omits a mechanism, its users build it — worse. Deprived of any way to declare or discover an interface, seven of eight agents hand-rolled an allgather of interface declarations within ninety seconds of one another, the integrator hand-rolled a published contract and two integration rounds, and every consumer hand-rolled runtime discovery by probing its producers. The result worked, and cost roughly double the code plus two defects located inside the probing logic itself. Treat independent reimplementations as the strongest signal a specification can receive about what it is missing — and note that it cuts both ways: the same evidence shows that the mechanisms we do provide are ones agents want, and that providing them is not the same as being necessary for the task.

L13. An interface invites a process; do not confuse the two. Our E2 harness prescribed the heaviest schedule the interface allows — agree a complete contract by agent-evaluated reduction before writing any code, then two synchronised integration rounds — and it cost $3 \times$ the wall time and $5.3 \times$ the context of an arm that used the same primitives opportunistically, for the same result. Having a collective for agreement does not mean agreement should be a phase. The mistake is ours, not the protocol’s, but it is the mistake a protocol makes easy, and it is worth naming: prefer opportunistic publication, and escalate to a full reduction only when it demonstrably fails.

L14. A conditional send plus an unconditional receive is a deadlock, and the protocol cannot save you from it. Our own E2 harness contained this bug. The integrator was told to send a fix assignment *for each failure*; the other ranks were told to receive one. When the test suite passed, there were no failures, so four ranks sat in a receive that nobody would ever satisfy. This is the oldest bug in message passing and `AGENTMPI` does not prevent it — no interface can, because the mistake is in the program. What it

did do is worth recording: the deadline-bounded receive turned an unbounded hang into a stall that `ampi status` localised in one command, naming the four ranks missing from the barrier and the exact tag they were waiting on; and because the receives remained posted, the ranks were recoverable rather than lost. The harness lesson is the same one MPI teaches about collectives with zero-length buffers: *send unconditionally, possibly empty*. An empty assignment is still an assignment.

L15. Schema evolution is a live concern. Adding a column to the journal mid-experiment broke a running 22-rank job. A durable protocol store must migrate on open; a journal written by an older runtime has to stay readable.

10 Discussion and limitations

Does this argue for more agents? No, and our own E2 result argues against over-claiming. The strongest objection to multi-agent systems is that splitting work disperses context and decision-making, and that one agent holding everything often does better. We think that objection is largely right, and that it is an argument about *mechanism*: it holds when the only tools available are ad-hoc message passing and prompt concatenation. AGENTMPI is a claim about what changes when information sharing, agreement and recovery are first-class — not a claim that more agents are better.

E2 sharpens this. Eight agents produced a working system either way, and the arm we ablated recreated the protocol’s mechanisms by itself. What the protocol bought there was not capability but *legibility*: after the run we can say how many operator applications the agreement took, in what tree, what the contract cost in tokens, which rank held which lock, and that 46 timed-out calls resumed rather than failed. That is worth something to a harness author, and it is much less than we would have claimed had the pass rates differed. It also tells us the honest scope of the contribution: AGENTMPI is for harnesses whose coordination you need to be able to specify and audit, not for proving that coordination is necessary.

Scale of the real-agent runs. A ten-concurrent-agent ceiling on our host bounded real-agent P well below the microbenchmark range. This is a limitation of the evaluation, not of the design, but it is a real one: we cannot report a 128-agent application run, and the selection thresholds in Section 5.2 are validated with stub executors. It is also the standard shape of an HPC evaluation — microbenchmarks at scale, applications at the scale the machine allows — and we have been explicit about which claims rest on which.

Silent failure is not solved. A5 includes plausible-but-wrong output, and AGENTMPI provides only mechanism for it: redundancy through `vote`, selection through `maxby`, attribution through window writer records. Detecting that a result is wrong remains the harness’s problem, as ABFT leaves verification to the algorithm [27]. We regard a protocol that claimed to solve it as overreaching.

The token is an imperfect unit. Thresholds denominated in tokens make the eager/rendezvous decision estimator-dependent, so two implementations may classify the same payload differently. This affects cost, not correctness, and we require implementations to report their estimator — but a protocol whose central threshold is approximate is worth flagging.

Interface advertisement should not have been omitted. Our strongest piece of evidence for a feature is one we left out. Section 8.2.2 reports that when the protocol offered no way to declare and discover an interface, seven of eight agents built the declaration half within ninety seconds of each other and every consumer built the discovery half by probing — at roughly double the code and with two defects inside the probing logic. A protocol whose users independently reimplement a mechanism, badly, is a protocol missing that mechanism. Appendix B of the specification lists capability advertisement and discovery among our deliberate omissions on the grounds that it is a research question; that judgement now looks wrong, and the specification records the evidence against it.

The interesting design question is not whether to add it but which form. The two arms demonstrate both: agreement-by-reduction, which produces a single artefact and can therefore be inconsistent with itself (Section 5.5); and discovery-by-probing, which cannot be inconsistent because it never decides anything globally, but which pushes the cost into every consumer. A protocol could offer a declaration operation — publish a typed interface into a window, keyed by provider — and let a consumer either read it or verify it against the provider’s actual behaviour, which is the useful union rather than a choice between them.

What we deliberately omitted. Following MPI’s practice of standardising only what is understood: automatic context compaction policy, semantic verification, cost-aware scheduling, inter-communicators, persistent collectives, partitioned communication, sessions, and capability discovery. Each is a research question, and a standard that answers one prematurely is worse than one that leaves a hook.

11 Related work

Classical agent communication languages. KQML [13] and FIPA-ACL [15] standardised *speech acts* with mentalistic semantics. They achieved little durable adoption, and the critiques are instructive: the semantics were unverifiable from observable behaviour, and the ontology commitment raised the cost of participation. AGENTMPI takes MPI’s opposite bet. What those systems got right and modern systems have largely lost is explicitness about interaction structure — FIPA’s Contract Net [40] is a named, analysable protocol in a way that “the orchestrator delegates to workers” is not — and AGENTMPI’s labelled collectives and declared topologies are an attempt to recover exactly that. Blackboard systems [12] anticipate our windows, and their central difficulty, control, is what `Win_fence` addresses. Tuple spaces [17] anticipate associative matching; MPI’s explicit addressing won in HPC for locality reasons that apply here too.

Modern agent protocols. MCP [36] standardises the agent-to-tool boundary; A2A [1] the agent-to-agent *task* boundary. Both are valuable and neither is a competitor: they occupy the transport and single-peer-delegation layers, and provide no group membership, no collectives, no consistency model for shared state and no failure model. AGENTMPI sits above them and could be carried over either.

Orchestration frameworks. AutoGen [47], LangGraph [2], CrewAI and the OpenAI Agents SDK are frameworks: they supply a coordination pattern (conversation, graph, crew, handoff) and the application is written inside it. LangGraph’s checkpoints and durable execution overlap our recovery briefing, and AutoGen’s actor rewrite overlaps our mailboxes. The difference is the same one that separated MPI from the frameworks of its era: a framework owns control flow, an interface does not. Our gather-returns-a-manifest rule and context ledger have, to our knowledge, no counterpart.

Research multi-agent systems. MetaGPT [26] defines a publish-subscribe message pool and standardised outputs; ChatDev [39] a chat chain over a waterfall process; CAMEL [32] role-playing dialogue. Each embeds a coordination mechanism in an application. Multi-agent debate [10] and self-consistency [46] are, in our terms, allreduce with `vote`; best-of- n is allreduce with `maxby`. That these fall out as operator choices rather than as architectures is some evidence the abstraction is at the right level. The MAST taxonomy [7] of multi-agent failures maps closely onto the failure classes of Section 6.

Parallel programming models. We borrow bulk synchrony from BSP [44] for window epochs, bounded staleness from SSP [24] for quorum collectives, lineage-based recovery from Spark [48] and durable execution for the recovery briefing, distributed object stores from Ray [37] for handles, and supervision trees from OTP [3]. We chose two-sided message passing plus one-sided windows over pure actors because actors have no collectives and no group naming, and over pure dataflow because agent work is not a static graph.

Context management. MemGPT [38] treats the context window as virtual memory to be paged, and PagedAttention [30] does the analogous thing for KV cache; both are OS analogies, where ours is a network one. Views are closer to derived datatypes than to paging: the harness declares which projection it wants rather than relying on a policy to evict. Long-context degradation [33] is a further argument for delivering less rather than more.

12 Conclusion

MPI’s history suggests that the way out of a proliferation of incompatible coordination layers is a thin, mechanism-level interface with a reference implementation, not another framework. We have tried to carry that answer into multi-agent systems, and found that MPI’s structure transfers almost intact while its cost model does not. Denominating transfer cost in tokens turns context exhaustion into flow control; making the reduction operator an agent makes serialised operator applications the dominant cost and inverts MPI’s algorithm selection in at least one standard case; and treating individual, frequent, sometimes-silent failure as normal requires ULFM plus leases, fencing epochs, join deadlines and replacement briefings. Two findings are, we suspect, more durable than the design. The first is the catalogue of defects that only real agents exposed: executors abandon retries, do not volunteer liveness, and cannot be trusted to keep program order. Each is an argument about where responsibility for reliability has to sit, and the answer is never the agent. The second is our own null result and what it took to see past it. Eight agents given these primitives but told nothing about how to coordinate built a contract window and per-rank interface declarations by themselves, and matched the prescribed schedule’s output for a third of its cost. That the abstraction is what agents reach for is the best evidence we have that it is the right abstraction; that they reach for it unprompted is a warning against selling it as a capability. It took an instrument that could not favour either arm — differential testing over generated programs — to find any difference at all, and what it found was not the negotiated contract pay-

ing off but the *synchronised integration round*: in the ablated arm a rank diagnosed a real specification violation, messaged the owner and the integrator twice with a one-line fix, and was never answered. What a protocol offers is not that coordination becomes possible, but that it becomes something a harness author can state, bound, and afterwards read back.

References

- [1] A2A Project, Linux Foundation. A2a protocol: An open protocol enabling communication and interoperability between opaque agentic applications, 2026. Current specification site. Technical Steering Committee includes AWS, Cisco, Google, IBM Research, Microsoft, Salesforce, SAP and ServiceNow; Apache-2.0; states the MCP/A2A division of labour.
- [2] Alice Labs. LangGraph tutorial 2026: Build stateful ai agents for enterprise, 2026. Secondary. Checkpointer tiers (SqliteSaver for development, PostgresSaver for production); `thread_id`-keyed independent state history; `interrupt_before/interrupt_after` with indefinitely persisted paused state.
- [3] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.
- [4] Wesley Bland, Aurelien Bouteiller, Thomas Herault, George Bosilca, and Jack Dongarra. Post-failure recovery of MPI communication capability: Design and rationale. *International Journal of High Performance Computing Applications*, 27(3):244–254, 2013.
- [5] Robert L. Bocchino, Vikram S. Adve, Sarita V. Adve, and Marc Snir. Parallel programming must be deterministic by default. In *Proc. 1st USENIX Workshop on Hot Topics in Parallelism (HotPar)*, 2009.
- [6] Mike Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 335–350, Seattle, WA, November 2006. USENIX Association.
- [7] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025. NOTE: verify author list and venue against the publisher record.
- [8] Tushar Deepak Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267, March 1996.
- [9] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.
- [10] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- [11] E. N. (Mootaz) Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, September 2002.
- [12] Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2):213–253, June 1980.
- [13] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)*, pages 456–463. ACM, 1994.
- [14] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
- [15] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Technical Report SC00061G, FIPA, 2002. <https://www.fipa.org/specs/fipa00061/SC00061G.html>.
- [16] Hubertus Franke, Peter Hochschild, Pratap Pattnaik, and Marc Snir. MPI on IBM SP1/SP2: Current status and future directions. In *Proceedings of the 1994 Scalable Parallel Libraries Conference*. IEEE Computer Society Press, 1994.
- [17] David Gelernter. Generative communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, January 1985.

- [18] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*, pages 202–210. ACM, 1989.
- [19] William Gropp, Ewing Lusk, Nathan Doss, and Anthony Skjellum. A high-performance, portable implementation of the MPI message passing interface standard. *Parallel Computing*, 22(6):789–828, September 1996.
- [20] William D. Gropp. Learning from the success of MPI. In Burkhard Monien, Viktor K. Prasanna, and Sriram Vajapeyam, editors, *High Performance Computing—HiPC 2001, 8th International Conference*, volume 2228 of *Lecture Notes in Computer Science*, pages 81–92. Springer, December 2001.
- [21] William D. Gropp. Parallel programming with MPI. Half-day tutorial, 2004. States that many parallel programs can be written with six MPI functions: MPI_INIT, MPI_FINALIZE, MPI_COMM_SIZE, MPI_COMM_RANK, MPI_SEND, MPI_RECV.
- [22] William D. Gropp and Ewing Lusk. Goals guiding design: PVM and MPI. In *Proceedings of the 2002 IEEE International Conference on Cluster Computing (CLUSTER 2002)*, pages 257–265, Chicago, IL, September 2002. IEEE Computer Society.
- [23] Naohiro Hayashibara, Xavier Défago, Rami Yared, and Takuya Katayama. The φ accrual failure detector. In *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems (SRDS 2004)*, pages 66–78. IEEE, 2004.
- [24] Qirong Ho, James Cipar, Henggang Cui, Jin Kyu Kim, Seunghak Lee, Phillip B. Gibbons, Garth A. Gibson, Gregory R. Ganger, and Eric P. Xing. More effective distributed ML via a stale synchronous parallel parameter server. In *Advances in Neural Information Processing Systems 26 (NIPS)*, pages 1223–1231, 2013.
- [25] Torsten Hoefler, James Dinan, Rajeev Thakur, Brian Barrett, Pavan Balaji, William Gropp, and Keith Underwood. Remote memory access programming in MPI-3. *ACM Transactions on Parallel Computing*, 2(2):9:1–9:26, 2015.
- [26] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352.
- [27] Kuang-Hua Huang and Jacob A. Abraham. Algorithm-based fault tolerance for matrix operations. *IEEE Transactions on Computers*, C-33(6):518–528, 1984.
- [28] Ken Kennedy, Charles Koebel, and Hans Zima. The rise and fall of High Performance Fortran: An historical object lesson. In *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages (HOPL III)*, pages 7–17–22, San Diego, CA, June 2007. ACM.
- [29] Martin Kleppmann. How to do distributed locking. Blog post, <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>, February 2016. Published 8 February 2016; accessed 2026.
- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [31] Ignacio Laguna, Ryan Marshall, Kathryn Mohror, Martin Ruefenacht, Anthony Skjellum, and Nawrin Sultana. A large-scale study of MPI usage in open-source HPC applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*, Denver, CO, November 2019. ACM.
- [32] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2303.17760.
- [33] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.
- [34] Message Passing Interface Forum. MPI: A message-passing interface standard. *The International Journal of Supercomputer Applications and High Performance Computing*, 8(3–4):165–414, 1994. Special issue on MPI.

- [35] Message Passing Interface Forum. MPI: A message-passing interface standard, version 5.0. Technical report, University of Tennessee, Knoxville, June 2025. Approved June 5, 2025; adds a standard Application Binary Interface.
- [36] Model Context Protocol Contributors. Model context protocol specification, revision 2026-07-28, July 2026. Stable release of the 2026-07-28 protocol revision; wire-incompatible with prior revisions.
- [37] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 561–577, 2018.
- [38] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [39] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. *arXiv:2307.07924*.
- [40] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, December 1980.
- [41] Marc Snir, Steve W. Otto, Steven Huss-Lederman, David W. Walker, and Jack Dongarra. *MPI: The Complete Reference*. MIT Press, Cambridge, MA, 1996. States that MPI-1 comprises 128 routines and gives the Forum’s rationale for the interface size (“Why is MPI so big?”). Online edition: <https://netlib.org/utk/papers/mpi-book/mpi-book.html>.
- [42] Rajeev Thakur, Ewing Lusk, and William Gropp. A case for using MPI’s derived datatypes to improve I/O performance. In *Proceedings of SC98: High Performance Networking and Computing*, November 1998.
- [43] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in MPICH. *International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.
- [44] Leslie G. Valiant. A bridging model for parallel computation. *Communications of the ACM*, 33(8):103–111, 1990.
- [45] David W. Walker. Standards for message passing in a distributed memory environment. Technical Report ORNL/TM-12147, Oak Ridge National Laboratory, August 1992.
- [46] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. *arXiv:2203.11171*.
- [47] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [48] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 15–28, 2012.